

Design and Implementation of an Interactive Human-in-the-Loop Sorting System over a Self-Healing Wireless Sensor Network

Muhammed Ayranci, Kemal Berkay Elçik, Andreea Musat
Technical University of Munich
Munich, Germany

Abstract—This report presents the design and implementation of an interactive, human-in-the-loop cargo sorting system operating on a self-healing wireless sensor network. The system is deployed across 7 nRF52840 development boards running the Contiki-NG operating system, combining embedded physical sensing, user interaction, and distributed wireless communication. Cargo packets are generated from force sensor measurements, classified by a human operator using an analog joystick, and delivered through a multi-hop network utilizing a distributed distance-vector routing mechanism. To guarantee reliability, the implementation incorporates a packet queuing system, periodic topology monitoring, and dynamic proxy routing, enabling communication to seamlessly continue despite node failures. The final demonstration validates the complete workflow—spanning cargo generation, multi-hop routing, real-time GUI visualization, delivery feedback, and fault recovery—proving the feasibility of integrating human decision-making into a robust industrial IoT application.

Index Terms—Wireless Sensor Networks, Contiki-NG, multi-hop routing, fault tolerance, industrial IoT, human-in-the-loop

I. INTRODUCTION

Wireless Sensor Networks (WSNs) enable distributed sensing and communication among resource-constrained embedded devices through cooperative multi-hop communication, making them well suited for industrial monitoring and cyber-physical systems [1]. Their ability to integrate sensing, computation, and wireless communication has led to their widespread adoption in Industrial Internet of Things (IIoT) applications [2].

This paper presents the design and implementation of a human-in-the-loop cargo sorting application developed for the Wireless Sensor Networks Laboratory. The system simulates an industrial sorting process in which cargo is generated from a physical force sensor measurement, assigned to a destination through operator joystick input, and delivered across a self-healing wireless sensor network. The implementation utilizes exactly 7 nRF52840 development boards running the Contiki-NG operating system, using NullNet for lightweight communication and standard CSMA/CA over IEEE 802.15.4 as the underlying wireless communication stack [2]–[5].

To support reliable operation, the system combines custom packet structures, distributed distance-vector routing, and dynamic fault recovery mechanisms that enable communication to continue despite node failures. The remainder of this paper presents the system architecture, communication protocol, human-in-the-loop workflow, fault-tolerance mechanisms, and the final system demonstration.

II. SYSTEM DESIGN

A. System Overview

The implemented system realizes an interactive cargo sorting application using a WSN composed of nRF52840 development boards. The application simulates an industrial process in which cargo generated from physical sensor data is assigned to a destination by a human operator and delivered through a multi-hop wireless network.

The process begins at the factory node, where a Force Sensitive Resistor (FSR) generates a cargo packet. The packet is transmitted to the operator node, where an analog joystick is used to select the delivery destination before forwarding it through the network. Once the cargo reaches the corresponding customer node, a report packet containing the delivery status and routing history is bounced back to the operator.

Communication is executed using the lightweight NullNet network layer provided by Contiki-NG, while medium access is handled by the CSMA/CA protocol operating over IEEE 802.15.4 on Channel 16. This lightweight stack enables application-specific packet handling while supporting reliable multi-hop communication between resource-constrained nodes [2], [3]. The overall system architecture is illustrated in Fig. 1.

B. Node Roles

The implemented application distributes its functionality across 7 active wireless sensor nodes, perfectly fulfilling the laboratory requirements. While all nodes execute the same Contiki-NG operating system, their application logic diverges according to their assigned roles.

The factory node (Node 7) initiates the sorting process by continuously sampling a Force Sensitive Resistor (FSR)

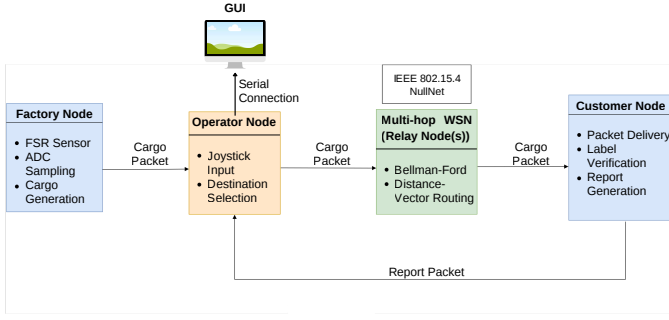


Fig. 1. System architecture of the implemented cargo sorting application.

via the analog-to-digital converter (ADC). After threshold confirmation, it generates a cargo packet containing a unique packet identifier, the measured force value, the corresponding ground-truth label, and the initial array required for subsequent routing before transmitting the packet to the operator node.

The operator node (Node 1) serves as the primary interface between the user and the network. It temporarily buffers incoming cargo packets, maps physical joystick movement to a destination label, and forwards the updated packet towards the selected customer node. The operator also receives return delivery reports, enabling real-time monitoring of routing paths and success rates.

The remaining 5 nodes operate as a fully decentralized mesh. While primary customer nodes (Nodes 2, 3, 4, and 5) represent the default delivery destinations and Node 6 acts as a dedicated relay, every single node in the network—including the factory and all customers—actively participates in the Bellman-Ford multi-hop routing. If any node receives a cargo or report packet destined for another device, it automatically acts as an intermediate relay and forwards it according to its local routing table, ensuring maximum physical flexibility and self-healing route discovery.

III. COMMUNICATION AND ROUTING

A. Network Stack

The implemented communication stack is based on Contiki-NG, using NullNet as the network layer and standard CSMA/CA as the medium access protocol over IEEE 802.15.4 Channel 16 [2], [3]. NullNet provides a lightweight communication interface without predefined routing functionality, allowing highly customized, application-specific packet handling and routing mechanisms to be implemented directly.

Each node is assigned a unique logical link-layer address derived from its identifier. Crucially, to force genuine multi-hop routing within a confined laboratory space, the radio transmission power on all nodes was intentionally reduced. This physical limitation ensures that nodes cannot communicate directly with the entire network, naturally establishing a multi-hop topology.

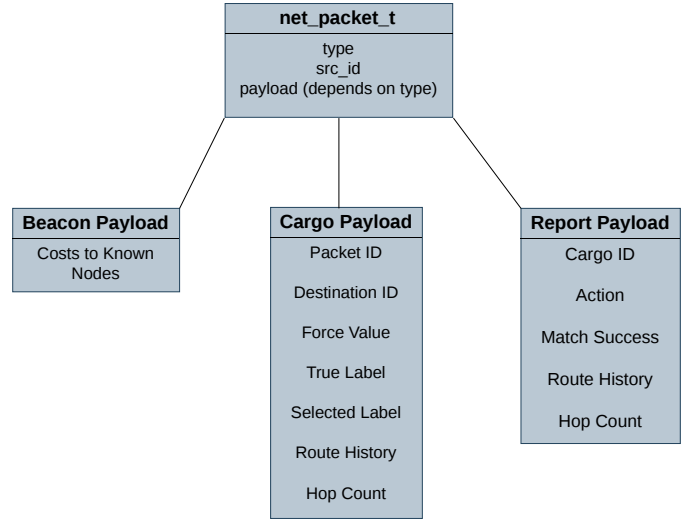


Fig. 2. Custom application packet structure. The header contains the packet type and source ID; the payload stores beacon, cargo, or report data.

B. Packet Structure

Communication is achieved using a custom packet format built on top of NullNet. The implementation defines three distinct packet types grouped under a single unifying data union: *Beacon*, *Cargo*, and *Report*.

Beacon packets periodically advertise routing costs to neighbouring nodes. Cargo packets carry the core application data, including the measured force value, true and selected labels, the destination identifier, and an array to log the cumulative routing history (limited to 10 maximum hops). Report packets are generated post-delivery (or upon packet drop) to return the final accuracy grade and completed forwarding path to the operator.

C. Multi-hop Routing

The network employs a distributed distance-vector routing mechanism based on the Bellman-Ford algorithm to enable multi-hop communication [1], [6]. Each node maintains a local routing table storing the path cost, next forwarding hop, and the timestamp of the most recent routing update for every reachable destination.

Routing information is exchanged via beacon packets broadcast every 3 seconds. Upon receiving a beacon, a node estimates the link quality using the Received Signal Strength Indicator (RSSI). The measured RSSI is converted into a tiered link penalty, categorizing connections from strong to weak. This penalty is mathematically combined with the advertised routing costs to calculate the optimal forwarding cost towards each destination.

When forwarding a cargo packet, a node independently selects the next hop from its local routing table, requiring no centralized coordination. This distributed approach enables the network to adapt to physical topology changes while maintaining reliable multi-hop delivery [1], [6].

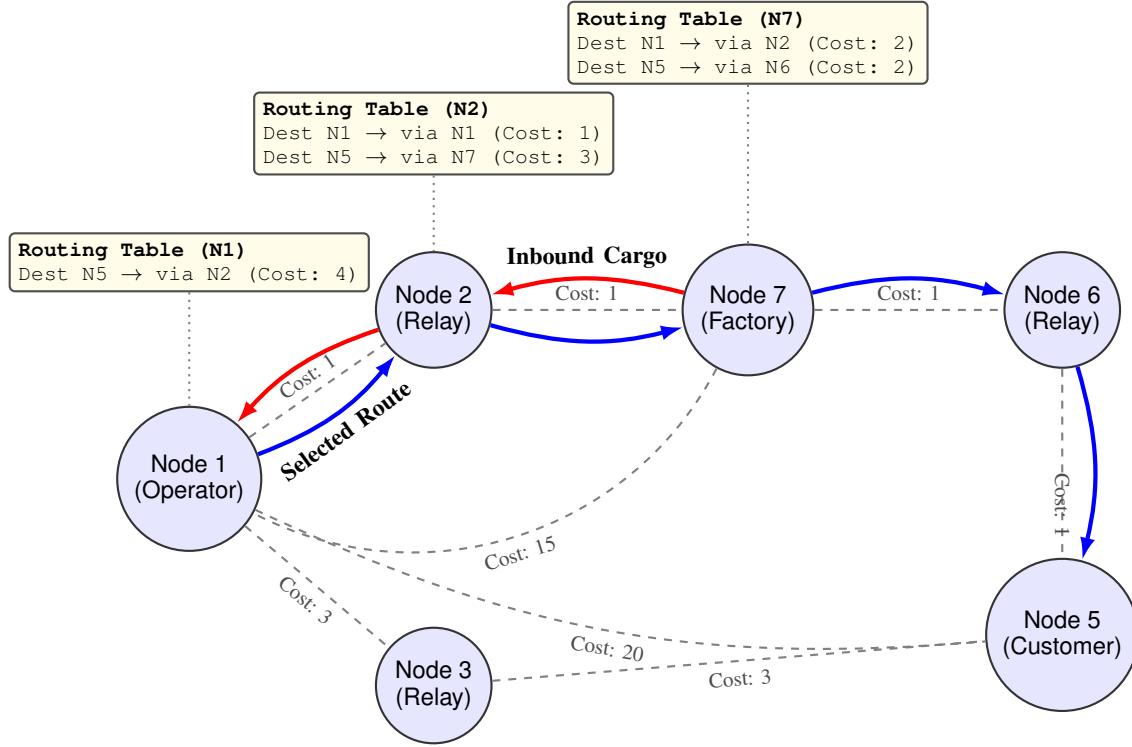


Fig. 3. Bidirectional multi-hop routing example. Due to network topology, the optimal outbound route (blue) uses the Factory (Node 7) as an intermediate relay based on Bellman-Ford metrics.

IV. HUMAN-IN-THE-LOOP OPERATION

The application tightly integrates physical sensing with user interaction. This workflow combines analog input, manual decision-making, and distributed communication within a single continuous cycle.

A. Cargo Generation

The cargo sorting process begins at the factory node, where the FSR is continuously polled. Once the physical input surpasses a specified pressure threshold, the measured force value is validated and classified into one of four predefined categories (A, B, C, or D) acting as the true label. By pressing the Button1, a cargo packet is generated containing this ground truth and is immediately forwarded to the operator node.

B. Operator Interaction

Because human response times are significantly slower than network transmission speeds, directly passing network packets to the operator input loop risks severe packet collision and loss. To resolve this, the operator node implements a circular buffer (queue) capable of holding up to 20 incoming cargo packets.

When a packet is dequeued, the operator node waits for the user to manipulate the analog joystick. The X and Y axes are mapped to the four destination labels. After a valid selection is made, the corresponding destination identifier is embedded into the cargo packet, and it is dispatched. The operator node then enters an active waiting state. If a predefined timeout occurs before a delivery report is received, it handles the failure by dropping the package.

C. Delivery Feedback

Upon reaching the destination customer node, the received cargo is evaluated. The node compares the operator-selected label with the ground-truth label initially assigned at the factory. The routing history accumulated during multi-hop forwarding is preserved and included in a report packet, alongside a binary success metric representing the delivery accuracy.

This report packet is dynamically bounced back toward the operator node by querying the local routing table. Once successfully received, the operator evaluates the success metric. If the delivery was accurate, the transaction is logged as resolved. However, if the delivery is reported as a failure (incorrect label), the operator automatically re-inserts the original cargo packet at the back of the circular buffer queue to be re-sorted by the user, provided there is buffer availability. Finally, the system proceeds to dequeue the next active package.

D. Graphical User Interface

To fulfill application requirements and visualize network operations in real-time, a custom Graphical User Interface (GUI) was developed using Python and the PyQt5 framework. The GUI interfaces with the operator node via an asynchronous serial connection. A dedicated background thread parses telemetry data—such as routing tables, queue status, and atomic delivery reports—without blocking the main event loop.

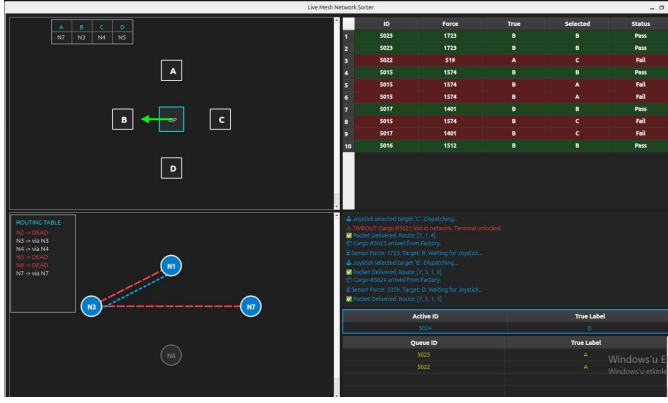


Fig. 4. Custom PyQt5 GUI visualizing network topology, cargo tracking, and delivery performance.

The interface is divided into key functional sectors. A status-driven Heads-Up Display (HUD) maps the customer labels (A-D) to specific physical nodes, dynamically updating when proxy routing is engaged. Concurrently, a live 2D topology map renders the distributed Bellman-Ford routing state. It visually traces the specific multi-hop paths of active cargo packets in real-time utilizing the path history array, and color-codes offline or dead nodes for immediate fault identification. Interactive tables on the right side display both the active circular buffer queue and the historical performance of sorted cargo.

V. FAULT TOLERANCE

To ensure reliable operation under changing physical network conditions, the implemented system incorporates mechanisms for failure detection and automatic route adaptation.

A. Failure Detection

Network connectivity is continuously monitored through the periodically transmitted beacon packets. Each node records the reception time of the latest beacon for every known destination. A parallel watchdog process periodically checks these timestamps. If a node fails to update its timestamp within a 10-second timeout window, the node is considered offline or dead. The corresponding routing cost is set to infinity, naturally preventing packets from being forwarded through the failed node in subsequent routing operations.

B. Dynamic Proxy Routing

To maintain application functionality during primary destination failures, the system supports dynamic proxy routing—a highly application-specific feature. The operator node continuously monitors the reachability of its primary customer nodes (Nodes 2-5).

If a primary customer node becomes unreachable, the application logic checks the routing table for available backup nodes. Because the network logic is highly decentralized, both the dedicated relay (Node 6) and even the Factory (Node 7) can dynamically transition into acting as proxy customers. The operator seamlessly remaps the physical joystick input from

the dead node to the newly assigned proxy node. Subsequent cargo packets are automatically redirected, and the proxy node will consume the cargo and generate delivery reports without modifying the application logic of the remaining network. Once the original node recovers, its primary role is restored.

VI. SYSTEM DEMONSTRATION

The implemented system was successfully validated by demonstrating the complete cargo sorting workflow on a physical network of 7 nRF52840 wireless sensor nodes. The demonstration verified the interaction between the factory, operator, relay, and customer nodes, covering the entire process from analog cargo generation to final delivery confirmation.

The demonstrated workflow successfully exhibited force sensor acquisition, packet buffering, destination selection via joystick input, forced multi-hop packet forwarding (via lowered TX power), and accurate delivery feedback. Furthermore, node failures were intentionally introduced to validate the fault-tolerance mechanisms. Unreachable nodes were successfully detected, and dynamic proxy routing immediately re-assigned unavailable customer nodes, allowing the interactive cargo delivery to continue without manual reconfiguration or software resets.

VII. CONCLUSION

This paper presented the design and implementation of an interactive, human-in-the-loop cargo sorting application operating on a self-healing wireless sensor network. The developed system successfully integrates embedded sensing, operator interaction, and distributed multi-hop communication within a lightweight Contiki-NG environment.

Custom packet structures, distributed distance-vector routing, circular packet buffering, and dynamic fault recovery mechanisms enable highly reliable cargo delivery while maintaining network operation despite physical node failures. The implemented workflow, monitored by a real-time PyQt5 visualization dashboard, demonstrates the successful integration of robust routing and user interaction into a single wireless sensor network application.

REFERENCES

- [1] H. Karl and A. Willig, *Protocols and Architectures for Wireless Sensor Networks*, John Wiley & Sons, 2005.
- [2] IEEE Computer Society, *IEEE Standard for Low-Rate Wireless Networks*, IEEE Std 802.15.4-2020, July 2020.
- [3] A. Dunkels, B. Grönvall and T. Voigt, “Contiki—A Lightweight and Flexible Operating System for Tiny Networked Sensors,” in *Proc. 29th Annual IEEE International Conference on Local Computer Networks (LCN)*, 2004, pp. 455–462.
- [4] Contiki-NG Community, “Contiki-NG: The OS for Next Generation IoT Devices,” [Online]. Available: <https://github.com/contiki-ng/contiki-ng>
- [5] Nordic Semiconductor, “nRF52840 Product Specification v1.7,” 2021. [Online]. Available: <https://infocenter.nordicsemi.com>
- [6] J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*, 8th ed., Pearson, 2021.